IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 2 August 2022, accepted 1 October 2022, date of publication 10 October 2022, date of current version 26 October 2022.

*Digital Object Identifier 10.1109/ACCESS.2022.3213030*

RESEARCH ARTICLE

# Forensic Recovery of File System Metadata for Digital Forensic Investigation

**JUNGHOON OH[1,2], SANGJIN LEE[1], AND HYUNUK HWANG[2], (Member, IEEE)**
[1]School of Cybersecurity, Korea University, Seoul 02841, South Korea
[2]The Affiliated Institute of ETRI, Daejeon 34044, South Korea

Corresponding author: Sangjin Lee (sangjin@korea.ac.kr)

**ABSTRACT** File system forensics is one of the most important elements in digital forensic investigations. To date, various file system forensic methods, such as analysis of tree structure and the recovery of deleted file data, have been studied. Among these file system forensic methods, the recovery of file system metadata is a key technique that makes digital forensic investigations possible by recovering metadata when it is not possible to obtain metadata in a regular manner because the file system structure is damaged due to an accident/disaster or cyber terrorism. Previous studies mainly focused on recovering record or entry data, which are the basic units of metadata, using carving techniques via a fixed values or values capable of range prediction at the beginning of the data. However, no studies have been conducted on metadata without such fixed values or values capable of range prediction. $LogFile, which is a metadata file of the New Technology File System (NTFS) that is one of the most used file systems at present, contains very important metadata that provide a history of all file system operations during a specific period. However, since there is no fixed value or a value capable of range prediction at the start position of the record, which is the basic unit of $LogFile, there have been no studies on recovery using record units, and only recovery by file and page have been possible. If the file header or page header of $LogFile is damaged, existing recovery methods cannot properly recover the metadata; in such cases, a record-level recovery method is required to recover the metadata. In this context, we investigated the mechanisms of record storage through a detailed analysis of the $LogFile structure and proposed a recovery method for records without fixed values. Our proposed method was implemented as a tool and verified through comparative experiments with existing forensic tools that recover $LogFile data. The experimental results showed that the proposed recovery method was able recover all the data that existing tools are unable to recover in situations where the $LogFile data were damaged. The implemented tools are released free of charge to contribute digital forensic community. Finally, we explained what important role $LogFile played in solving real-world cases and confirm the importance of recovering $LogFile data in situations where file systems may be damaged due to accidents and disasters.

**INDEX TERMS** File system, forensics, metadata, forensic recovery.

## I. INTRODUCTION

File system forensics is a branch of digital forensics that analyzes the structure of a file system and examines its contents [1]. Because most of the evidence that can be obtained during a digital forensic investigation is stored in file units, file system forensics is the most basic and important factor for forensic investigators. Various forensic methods, such as analysis of tree structure [1], [2], [3], [4] and the recovery of deleted file data [5], [6], [7], have been studied to find important evidence in file systems. Among these file system forensic methods, the recovery of file system metadata is a key technique that makes digital forensic investigations possible by recovering metadata when it is not possible to obtain metadata in a regular manner because file system structure is damaged due to an accident/disaster or cyber

The associate editor coordinating the review of this manuscript and approving it for publication was Aniello Castiglione.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

terrorism [8], [9]. Additionally, recovering file system metadata from unallocated areas can help to discover additional evidence [10].

Previous studies mainly focused on recovering record or entry data, which are basic units of metadata, using carving techniques via a fixed value or a value capable of range prediction at the beginning of the data [10], [11], [12], [13], [14], [15], [16]. Indeed, no studies have been conducted on metadata without such a fixed or a value capable of range prediction.

$LogFile, a metafile of the New Technology File System (NTFS), is a log file that records data from such file system transactions as file creations, deletions, data changes, and name changes. Transaction data are stored in record units, where each record contains "redo" data and "undo" data. If the file system is broken due to a system error, such as a sudden power outage, the operating system restores the file system to a normal state by using the "redo" data and "undo" data of records in $LogFile [1]. Therefore, $LogFile is an important artifact that provides information about all file operations in the file system and history of file data changes during a specific period [17], [18], [19].

If a Windows system that uses NTFS as the main file system is abnormally stopped due to an accident/disaster or a cyber terrorism, analyzing $LogFile can help identify the file that was created before the system stopped and exactly when it stopped. However, the file system in such cases can be damaged arising from physical damage to the storage device or anti-forensic behavior, thereby causing a situation where the $LogFile data cannot be acquired and analyzed normally.

Previous studies regarding the recovery of $LogFile data have been conducted on file unit recovery [15] and page unit recovery [16]. If a part of a $LogFile, such as a file header or a page header, is corrupted, there is a limitation in these data recovery is not performed properly. In order to recover the data as much as possible even when the $LogFile data are damaged, the recovery of record units is necessary, but since the starting position of the $LogFile record does not have fixed values nor a value capable of range prediction, there has been no previous research on the recovery of record units. In this context, we investigate the mechanisms of record storage through a detailed analysis of the $LogFile structure and propose a recovery method for records without fixed values. This method is implemented as a tool and verified through comparative experiments with existing forensic tools that recover $LogFile data.

This study makes the following contributions to the literature:

- We introduced an algorithm to recover $LogFile records based on record carving. To the best of our knowledge, this paper is the first systematic study regarding the recovery of $LogFile record data.

- The proposed method recovers all records which are not damaged, even if it is not possible to obtain record data in a regular manner, and makes the existing techniques available, such as file-level event generation [17] and tracking file data history [19].

- This paper provided a comparative performance evaluation of our method relative to well-known forensic tools developed to recover $LogFile data.

- The implemented tools are released free of charge to contribute to the digital forensic community.

The remainder of this paper is organized as follows: Section 2 introduces previous studies, and Section 3 explains the need to recover $LogFile data by using specific examples. Section 4 introduces the internal structure of $LogFile. Section 5 details how to recover $LogFile data based on the record carving technique. Section 6 introduces the developed tools and experimentally evaluates their performance in comparison with currently available tools. In Section 7, we introduce a case study involving the use of the proposed tool. Finally, Section 8 summarizes the conclusions of this study.

## II. RELATED WORKS

Previous studies on the recovery of file system metadata have been mainly conducted on NTFS, APFS, and Ext4 file systems, which are the main file systems of the most commonly used operating systems. In addition, studies on the recovery of metadata using timestamps have also been conducted.

### A. RECOVERY OF NTFS METADATA

#### 1) $MFT

Fuchs [11] performed a study on recovering data of master file table ($MFT), the metadata for storing information on files and directories in NTFS, to entry units, which is the basic storage unit. The recovery of an entry was done by carving the entry using "FILE", which is a fixed value located in front of the entry. Fuchs studied how to extract the file name and timestamp from a recovered $MFT entry, and recovery of the broken entry was also conducted. The tools of the study were implemented and released as MFTEntryCarver. Additionally, Fuchs speculated that the reason why a large number of $MFT entries remain in unallocated areas is that a part of the memory is saved to the disk in the form of a crash dump file and then deleted and left.

#### 2) $USNJRNL

Oh [10] researched recovering $UsnJrnl data, the metadata that stores the file change history of NTFS, to record units, which is a basic storage unit. The recovery of $UsnJrnl record data was performed by carving records using the version information and the predicted value of the record size located at the beginning of the record; additional predictable field values were checked to minimize false positives. He added a researched recovery function to the NTFS Log Tracker [17] and released it. In addition, he tested how many $UsnJrnl records were recovered from unallocated area in hard disks, and it was confirmed that change journal data up to 11 months old could be obtained.

#### 3) $INDX

Segev [12] studied how to recover deleted index entries remaining in the slack area of the $INDX attribute, which stores directory index data of NTFS. The recovery of an entry was carried out using predictable values of the $FILE_NAME attribute in an entry, and the file name and timestamp information of the deleted file could be extracted from the $FILE_NAME attribute in the recovered index entry. This allowed for additional acquisition of traces of deleted files that could not be obtained from $MFT data. He implemented the researched recovery function as INDXRipper and released it.

#### 4) $LOGFILE

Previous studies on $LogFile, another type of metadata that stores file change history in NTFS, conducted only for the recovery of file units and page units. The X-Ways Forensics [15] provided the recovery of $LogFile data to file units. The recovery of the file was perform using the method of carving the files using the fixed value "RSTR" of the restart page located at the beginning of $LogFile file. However, if the restart page in $LogFile was corrupted or the logging page was corrupted, the recovery of the file could not be performed properly. Bulk Extractor, developed by Garfinkel [16], recovered the restart page and the logging page of $LogFile, and provided a function for storing each recovered page in one file. The recovery of a page was achieved by carving pages using "RSTR" and "RCRD", which are fixed values of the frontmost portion of each page. However, if the header of the page was damaged, the recovery of a page could not be performed properly.

### B. RECOVERY OF APFS METADATA

Plum [6] studied the recovery of deleted files by restoring the metadata of APFS, such as superblocks and volume superblocks. Each recovery of metadata used a method of carving blocks using "NXSB" and "APSB", which are fixed values at the front of the data. The metadata recovery method in the study was performed for the recovery of deleted files, but the recovery method could also be sufficiently utilized to build a file system tree even when the file system could not be analyzed normally due to damage of the file system. In addition, LSoft Technologies [13] revealed a method of recovering nodes of the B-tree used to construct the file system tree through a carving technique. This method could be used to recover metadata of files and directories.

### C. RECOVERY OF Ext4 METADATA

Dewald [14] studied how to construct a file system tree by recovering inodes, which store metadata in files/directories, even if the superblock or group descriptor table of the Ext4 file system is damaged and the file system cannot be analyzed. The recovery of inodes was achieved using a method of carving by creating a pattern through predictable values such as access rights, timestamps, and link counts rather than a fixed value. The proposed recovery method was implemented as a module in Sleuthkit framework and released.

### D. RECOVERY OF TIMESTAMP

Nordvik [20] performed a study to recover metadata using the characteristics of timestamps included in the metadata of each file system. The recovery method looked for corresponding patterns using a feature in which the timestamp of the same value was continuously stored, and then carving of the metadata including the pattern was performed. In this study, Nordvik performed tests to recover $MFT entries of NTFS and inodes of Ext4 using the proposed recovery method. Porter [21] improved Nordvik's research by recovering metadata using a continuous pattern of similar timestamps with the same prefix. This allowed the recovery accuracy to be improved by recovering additional metadata not found in Nordvik's original approach.

## III. THE NEED FOR RECOVERY OF $LogFile DATA

### A. ACCIDENT/DISASTER

If a Windows system stops suddenly due to an accident/disaster, data on the $MFT entry stored in the memory may not be saved to the disk due to the file caching mechanism of the Windows system [22]. A representative example of this situation is the sinking of the MV Sewol ferry in South Korea in 2014 [8]. In the accident, the Windows XP-based CCTV system of the ferry stopped working as it sank, but the exact time at which it had stopped and data from the last CCTV video that had recorded the capsizing could not be obtained using only data on the MFT entry left on the disk.

In this case, analyzing the $LogFile data can help determine the exact time at which the system stopped and the location of data from the last CCTV video. This is because the $LogFile immediately stores data for file system recovery on the disk [23].

However, fires and flooding caused by accidents/disasters can physically damage disks of the system such that even if the data are recovered, the normal structure of the file system or the file data may be damaged.

Therefore, even in such a case, there is a need for a method to recover the $LogFile data.

### B. CYBER TERRORISM

Cyber terrorism refers to the act of interrupting or destroying systems or networks for political purposes by using the Internet [24]. A cyber terrorism against a Windows system can destroy the metadata of the file system so that it no longer operates. A representative example is the "Dark Seoul" campaign, a cyber terrorism that occurred in South Korea in 2013 [9].

The malware used in the DarkSeoul campaign carried out an attack that overwrites the front of the volume, including the MBR (Master Boot Record) and the VBR (Volume Boot Record), with specific value ("PRINCPES" or "HASTATI") [25]. In such a case, analyzing the $LogFile can help obtain

traces of malware and file data used before the system was destroyed and stopped [17].

However, the method of disk destruction used in the Dark-Seoul campaign sometimes damaged some of the $LogFile data located in front of the NTFS volume. In this case, the data could not be obtained using the existing recovery methods [26].

Therefore, it is important to recover $LogFile data even if $LogFile is not accessible in a normal manner through file system analysis. Moreover, even if the $LogFile data are corrupted, it is important to recover data from an intact area.

## IV. INTERNAL STRUCTURE OF $LogFile

The $LogFile consists of pages that are 4096 bytes in size. The pages are divided into a restart area and a logging area [1]. (See. Fig. 1.)

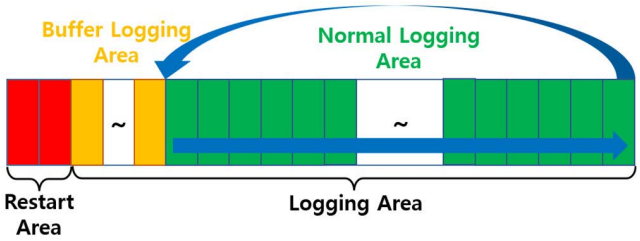



**FIGURE 1.** Structure of $LogFile.

The restart area is composed of two consecutive pages at the beginning of the file, and the second page is used for backup [1]. Each page header in the restart area begins with an "RSTR" magic number, and stores the LFS (LogFile Service) version by dividing it into a MinorVersion field and a MajorVersion field. Typically, LFS version 1.1 is used by operating systems prior to Win10 and version 2.0 is used for those subsequent to it [27]. Additionally, the FileSize field in the restart page header stores the $LogFile size [23]. (See. Fig. 2.)

| 0 1 2 3 | 4 5 | 6 7 | 8 9 | A B | C D | E F |
|---|---|---|---|---|---|---|
| "RSTR" (Magic Number) | Update Sequence Offset | Update Sequence Count | CheckDiskLSN | | | |
| SystemPageSize | LogPageSize | | Restart Offset | Minor Version | Major Version | |
| UpdateSequenceArray | | | | | | |
| CurrentLSN | | | LogClient | ClientFree List | ClientIn UseList | Flags |
| SeqNumberBits | RestartArea Length | ClientArray Offset | FileSize | | | |

**FIGURE 2.** Header of the restart page.

The logging area is divided into a buffer logging area and a normal logging area, and stores records that contain transaction data. The buffer logging area is an area where records are temporarily stored before being stored in the normal logging area. Therefore, the most recent records are stored in this area [17]. The buffer logging area is composed of the first two pages in case of LFS 1.1 and the first 32 pages in case of LFS 2.0 [27].

The remainder of the area excluding the buffer logging area is the normal logging area. Records are sequentially stored; when it is full, it is once again stored at the front of the normal logging area [1]. (See. Fig. 1.)

A page in the logging area consists of a header and multiple records. Each record stores the operation content for MFT entry, and multiple records are sequentially gathered to form a single transaction. The first and middle records of a transaction are update records, and the last record is a commit record [1]. The record consists of header, redo data, and undo data. This LSN field of the record header stores the LSN (Log Sequence Number) indicating its order and the Previous LSN field stores the LSN of the previous record. (See. Fig. 3.) The details of each field of the record header are shown in Appendix [17].

| 0 1 2 3 | 4 5 | 6 7 | 8 9 | A B | C D | E F |
|---|---|---|---|---|---|---|
| This LSN | | | Previous LSN | | | |
| Client Undo LSN | | | Client Data Length | | Client ID | |
| Record Type | Transaction ID | | Flags | Alignment or Reserved | | |
| Redo OP | Undo OP | Redo Offset / Redo Length | Undo Offset | Undo Length | Target Attribute | LCNs to follows |
| Record Offset | Attr Offset | MFT Cluster Index / Alignment or Reserved | Target VCN | | Alignment or Reserved | |
| Target LCN | | Alignment or Reserved | | | | |

**FIGURE 3.** Header of record.

The LSN is 64 bits in size, and consists of a sequence number and an offset number. The sequence number is a value that increases by one each time a normal logging area is toured, and the offset number is the value used to obtain a record offset representing the location of the corresponding record based on the start location of the $LogFile. The record offset of the corresponding record can be obtained by multiplying the offset number by eight. The sequence number is located at the top of the LSN, and has the number of bits corresponding to the SeqNumberBits field of the restart page header. The offset number is located at the bottom [27]. Fig. 4 shows the structure of the LSN.

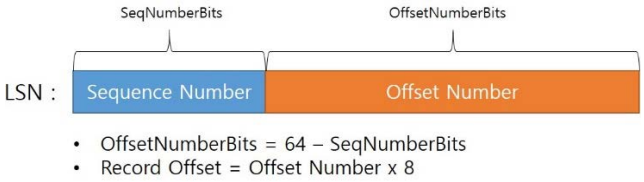



**FIGURE 4.** Structure of LSN. (Log Sequence Number.)

The redo and undo data of the record are used to perform operations corresponding to the redo/undo OP of the record header, and are stored with a size corresponding to the redo/undo length of the header in the position corresponding to the redo/undo offset of the header [17].

The header of the logging page starts with an "RCRD" magic number, and stores the largest LSN among the records,

including the record going over the page within the page, in the "Last LSN" field. The largest LSN among records that do not spill over the page is stored in the "Last End LSN" field of the header. The detailed structure of the logging page header is shown in Fig. 5 [17].

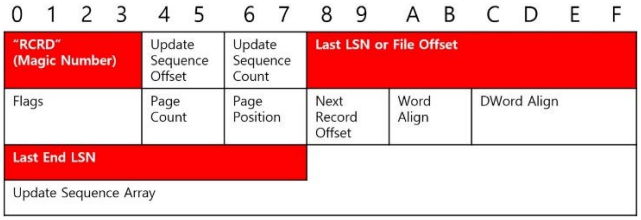



**FIGURE 5.** Header of page in logging area.

## V. RECOVERY METHOD

### A. BASIC CONCEPT OF RECORD CARVING

If the $LogFile has been damaged, a record-based method of recovery is required to obtain as much data from it as possible. However, because the $LogFile record does not have a fixed value, such as a magic number, information on the signature that can determine whether a given record is a $LogFile record must be generated for record carving.

The "This LSN" field located at the forefront of the $LogFile record stores the LSN of the current record. Therefore, the "This LSN" value cannot be used as a magic number for carving because it has different for each record. On the contrary, because all records in a page have the same sequence number, the sequence number that can be extracted from the "This LSN" value can be used as a magic number for carving records located in one page. The sequence number that all records in the page have in common can be obtained from the Last LSN value of the logging page header. For this, the OffsetNumberBits value is first obtained from the SeqNumberBits value obtained from the restart page header. The sequence number can then be obtained by performing a right-shift operation on the Last LSN value of the logging page header by using the OffsetNumberBits value.

$$\text{OffsetNumberBits} = 64 - \text{SeqNumberBits}$$

$$\text{Sequence Number} = \text{PageHeader.LastLSN} \gg \text{OffsetNumberBits}$$

If sequence number is obtained as described above, the signature consisting of the conditions provided in Table 1 can be generated for record carving. Record carving is then performed by searching for the data on the logging page in units of eight bytes, finding a start location for the record that meets the conditions of the record carving signature, and identifying the size of the record through the Client Data Length value of its header and analyzing it.

### B. OVERVIEW OF RECOVERY OF $LogFile DATA

The overall process of $LogFile data recovery is as follows: The pages, which are the basic components of the $LogFile, are first restored by carving with the magic number of the page header. The structure of the $LogFile is then determined by analyzing the restart and the logging pages recovered from the page carving operation. Once this structure has been determined, the scopes of the buffer logging area and the normal logging area can be accurately known, and the recovered logging pages can be classified into buffer and normal logging pages. The structure of the $LogFile should be identified in this way because the appropriate recovery technique can subsequently be applied to each area and logging page by knowing the exact scopes of each logging area and type of logging page. Following this, logging pages that are uncarved due to damage to the header are recovered in each logging area because there is a high likelihood that the $LogFile data remains even on uncarved pages. The uncarved pages are recovered by creating a virtual header through a record carving technique according to the characteristics of each logging area. Then, sorting work is performed for all logging pages, including the recovered ones, because the pages must be analyzed to avoid missing records. Finally, record carving is performed on the sorted logging pages to obtain the $LogFile data. Fig. 6 is a summary of the entire process of recovering $LogFile data through record carving.

We now detail the process of recovering the $LogFile data.

**TABLE 1.** Signature for record carving.

| No | Conditions |
|---|---|
| 1 | This LSN >> OffsetNumberBits == Sequence Number |
| 2 | (Previous LSN == 0) or<br>(Previous LSN >> OffsetNumberBits == Sequence Number) |
| 3 | (Client Undo LSN == 0) or<br>(Client Undo LSN >> OffsetNumberBits == Sequence Number) |
| 4 | (Record Type == 0x1) or (Record Type == 0x2) |

### C. STEP 1: CARVING $LogFile PAGES

The first task when recovering $LogFile data is to carve the pages. Page carving is performed by using the magic numbers of the restart page ("RSTR") and the logging page ("RCRD"), and by using values that are fixed in each page header. Table 2 shows the header signature for carving the $LogFile page.

The range of page carving operations can be divided into two cases. First, when the restart page is carved, the carving operation starts at the location at which the first restart page was carved and ends at the start location plus the size of the $LogFile. The size of the $LogFile can be obtained from the header of the carved restart page. This method is feasible because the $LogFile file is generally created without fragmentation during the formatting of the file system. Second, if the restart page is not carved, page carving is performed only over a range of size of 64 MB, starting from the location where the first logging page was carved. This method is used because the $LogFile increases in size depending on the volume, but only up to 64 MB. Therefore, unless the user directly adjusts its size, the size of the $LogFile in the volume is generally 64 MB.

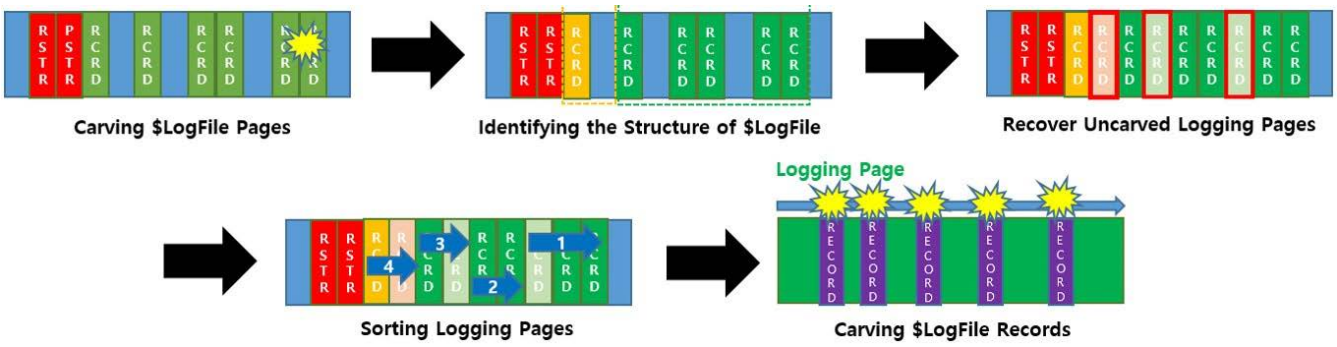



**FIGURE 6.** Overview of recover $LogFile data.

**TABLE 2.** Header signature for page carving.

| Type | Signature |
|---|---|
| Restart Page | Magic Number== "RSTR" |
| | Update Sequence Offset == 0x001E |
| | Update Sequence Count == 0x0009 |
| | Check Disk LSN == 0x0000000000000000 |
| | System Page Size == 0x00001000 |
| | Log Page Size == 0x00001000 |
| Logging Page | Magic Number == "RCRD" |
| | Update Sequence Offset == 0x0028 |
| | Update Sequence Count == 0x0009 |

When page carving is performed by specifying the range in the above way, it is not necessary to perform page carving for the entire volume such that the time needed can be greatly reduced.

### D. STEP 2: IDENTIFYING THE STRUCTURE OF $LogFile

Once page carving is complete, we need to determine the structure of the $LogFile. If the ranges of the buffer logging area and the normal logging area as well as the types of carved logging pages can be accurately identified using knowledge of the structure of the $LogFile, an appropriate recovery technique can be applied to the logging pages in each area. The logging pages classified in this operation are stored in the list of buffer logging page and the list of normal logging page, respectively.

The structure of the $LogFile depends on the LFS version, which can be obtained from the header of the restart page. Therefore, the method used to determine the structure of the $LogFile varies as described below, depending on whether the restart page is carved and the number of carved restart pages.

#### 1) IN CASE ALL RESTART PAGES ARE CARVED

If both restart pages are carved, the start position of the $LogFile data is fixed such that the ranges of the buffer logging area and the normal logging area can be accurately identified. Table 3 shows ranges of the buffer logging area and the normal logging area based on the location of the restart page for each LFS version. PFRP stands for the "position of the first restart page" and PSRP stands for the "position of the second restart page." The size of the $LogFile can be obtained from the restart page header.

In this way, each logging page recovered through carving in the previous step can be classified into a buffer logging page or a normal logging page according to the area information in the Table 3.

**TABLE 3.** Range of buffer/normal logging area.

| LFS Version | Logging Area | Area Range |
|---|---|---|
| LFS 1.1 | Buffer | (PFRP + 0x2000) ~ (PFRP + 0x4000) |
| | | (PSRP + 0x1000) ~ (PFRP + 0x3000) |
| | Normal | (PFRP + 0x4000) ~ ((PFRP + $LogFile Size) – 0x1000) |
| | | (PSRP + 0x3000) ~ ((PSRP + $LogFile Size) – 0x2000) |
| LFS 2.0 | Buffer | (PFRP + 0x2000) ~ (PFRP + 0x22000) |
| | | (PSRP + 0x1000) ~ (PFRP + 0x21000) |
| | Normal | (PFRP + 0x22000) ~ ((PFRP + $LogFile Size) – 0x1000) |
| | | (PSRP + 0x21000) ~ ((PSRP + $LogFile Size) – 0x2000) |

#### 2) IN CASE ONE RESTART PAGE IS CARVED

If only one restart page is carved, we should first determine whether the carved restart page is the first or the second. If the order can be determined, the range of each logging area is identified through the contents of Table 3.

Two methods can be used to determine the order of carved restart pages: by using the difference between a buffer logging page and a normal logging page for each LFS version, and by using the adjacency in physical location between the restart page and the buffer logging page.

The method that uses the difference between the buffer logging page and the normal logging page for each LFS version is as follows: In the case of LFS 1.1, the Last LSN field in the header of the buffer page stores a file offset value, instead of the Last LSN value. The file offset value indicates the location of the normal logging page from which data on the corresponding buffer logging page are copied from within the $LogFile. Thus, the file offset value of the buffer logging page is always a multiple of the page size (typically $0 \times 1000$). On the contrary, the Last LSN value of the header of the normal logging page cannot be a multiple of the page size because if this is the case, the record offset obtained from the LSN is also a multiple of the page size. This means that the record with the LSN corresponding to the "This LSN" value is located in the header, which is the starting position of the page. Such a case is not possible. In case of LFS 1.1, the file offset value of the header of the buffer logging page is thus a multiple of page size and the Last LSN of the normal logging page header is not. Therefore, the order of the restart pages can be determined by checking whether such a difference exists between the logging pages at the point determined to be the boundary between the buffer logging area and the normal logging area.

The method used in LFS 1.1 cannot be used in case of LFS 2.0 because all buffer logging pages have the Last LSN value in their headers. In this case, we can use the fact that the record offset value calculated from the Last End LSN of the header of the buffer logging page does not indicate the area in which the corresponding buffer logging page is located, unlike in case of the normal logging page. This method is possible because the buffer logging page contains data to be copied to the normal logging page, and thus the record offset calculated through the Last End LSN of the header of the buffer logging page is the address of the area of the normal logging page to be copied later, and not the address of the area where the current buffer logging page is located. In case of LFS 2.0, the record offset value calculated through the Last End LSN of the header of the buffer logging page thus does not point to an area of its own page, but that calculated through the Last End LSN of the header of the normal logging page does. Therefore, as in case of LFS 1.1, the order of the restart pages can be determined by checking whether such a difference exists between the logging pages at the point determined to be the boundary between the buffer logging area and the normal logging area.

The second method to determine the order of the carved restart pages uses the adjacent physical positions of the buffer logging page and the restart page. Given that the buffer logging page is located immediately after the second restart page, the carved restart page can be determined to be the second restart page if the logging page is carved immediately after the carved restart page. This method can be used regardless of the version of LFS.

If the order of restart pages cannot be determined through the above two methods, the range of each logging area should be identified regardless of this order. The continuity of normal logging pages can be used to this end. The relevant method is based on the fact that the offset number calculated through the Last LSN of the header remains the same or continuously increases as the physical address increases in the normal logging area, but not in case of the buffer logging area. Therefore, while checking the offset number generated through the Last LSN value of the header of the carved logging pages in turn, we check whether this value is the same or has increased—over three pages in case of LFS 1.1 and over 33 pages in case of LFS 2.0. If such continuous page flow is obtained, the first page is considered to be the start page of the normal logging area. All logging pages at the back, including the corresponding page, are classified as normal logging pages, and all logging pages at the front are classified as buffer logging pages.

Algorithm 1 is a pseudocode that identifies the range of each logging area and classifies logging pages when only one restart page is carved by using the methods described above.

### 3) IN CASE NO RESTART PAGE IS CARVED

If no restart page is carved, it is necessary to first check the similarity in data between the buffer logging pages in LFS 1.1. The first and second logging page located immediately after the restart page are buffer logging pages in LFS 1.1. These two pages contain data before and after the operational data of the record are applied to the $MFT entry. The data on both pages are thus the same except for the last record (see Fig. 7).

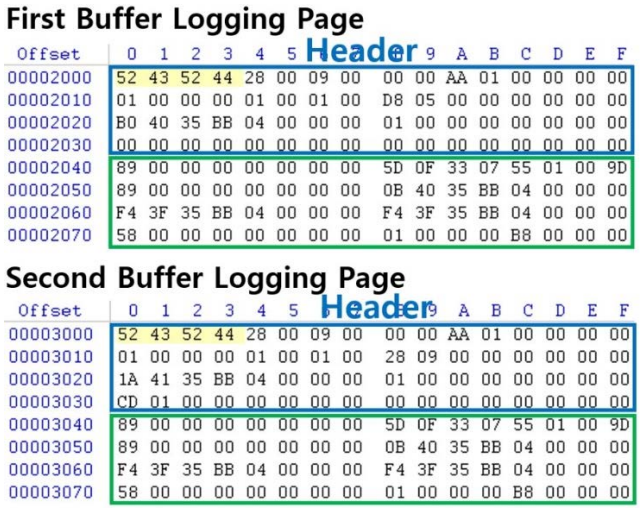



**FIGURE 7.** Buffer logging pages in LFS 1.1.

Therefore, if the first and second logging pages that are carved are physically adjacent to each other, and 64 bytes of data after the header of the two pages are the same, the LFS can be identified as version 1.1. In this case, the locations of the two buffer logging pages can be known, and thus all logging pages carved behind the two buffer logging pages can be classified as normal logging pages.

**Algorithm 1** Determining the Structure of $ LogFile When One Restart Page Is Carved

```
if LFS version == 1.1 then
    if there are carved logging pages at +0x3000 and +0x4000 from
      carved_restart_page_addr then
        result_1 ← 0x3000_page_last_lsn % 0x1000
        result_2 ← 0x4000_page_last_lsn % 0x1000
        if result_1 == 0 && result_2 != 0 then
            carved restart page is first restart page
            buffer_logging_area_start_addr ← carved_restart_page_addr+0x2000
            normal_logging_area_start_addr ← carved_restart_page_addr+0x4000
        else
            carved restart page is second restart page
            buffer_logging_area_start_addr ← carved_restart_page_addr+0x1000
            normal_logging_area_start_addr ← carved_restart_page_addr+0x3000
        end if
    else
        if first carved logging page is located after carved restart page then
            carved restart page is second restart page
            buffer_logging_area_start_offset ← carved_restart_page_offset+0x1000
            normal_logging_area_start_offset ← carved_restart_page_offset+0x3000
        else
            Find 3 consecutive pages with the same or increasing offset_number
            from last_lsn
            normal_logging_area_start_addr ← detected_page_addr
            all carved pages are buffer pages before normal logging area
        end if
    end if
else if LFS version == 2.0 then
    if there is carved logging pages which located at +0x21000 from
      carved_restart_page_addr then
        temp_value ← 0x21000_page_last_end_lsn << sequence_number_bits
        offset_number ← temp_value >> sequence_number_bits
        record_offset ← offset_number × 8
        if record_offset > 0x22000 && record_offset < 0x23000 then
            carved restart page is second restart page
            buffer_logging_area_start_addr ← carved_restart_page_addr+0x1000
            normal_logging_area_start_addr ← carved_restart_page_addr+0x21000
        else
            carved restart page is first restart page
            buffer_logging_area_start_addr ← carved_restart_page_addr+0x2000
            normal_logging_area_start_addr ← carved_restart_page_addr+0x22000
        end if
    else
        if first carved logging page is located after carved restart page then
            carved restart page is second restart page
            buffer_logging_area_start_addr ← carved_restart_page_addr+0x1000
            normal_logging_area_start_addr ← carved_restart_page_addr+0x21000
        else
            Find 33 consecutive pages with the same or increasing offset_number
            from last_lsn
            normal_logging_area_start_addr ← detected_page_addr
            all carved pages are buffer logging pages before normal logging area
        end if
    end if
end if
```

If each logging area cannot be identified using the above method, the method that uses continuity of the normal logging pages, described above, should be used. While checking the offset number value generated through the Last LSN value of the header of the carved logging pages in turn, we check whether this value is the same or has increased over 33 pages. If such continuous page flow is obtained, the first page of the flow is considered to be the start page of the normal logging area.

If the restart page is not carved, the sequence number cannot be acquired and the record carving signature cannot be generated through the OffsetNumberBits value because the SeqNumberBits value cannot be obtained from the header of the restart page. Therefore, a method to obtain the OffsetNumberBits value is required even in this case. One such method uses the difference in the sequence numbers of the logging pages when there is a circulation in the normal logging area. The difference between the sequence number from the smallest Last LSN and that from the largest Last LSN is one in this case (see Fig. 8).

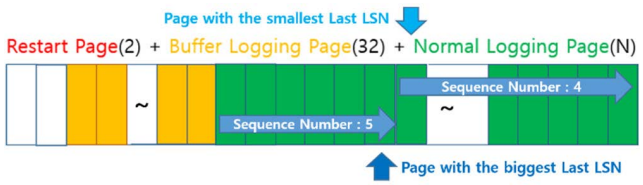



**FIGURE 8.** Status of sequence numbers in case of rotation.

Therefore, with respect to the smallest and the largest Last LSN values, we calculate the difference between the sequence numbers in each case while increasing the OffsetNumberBits value from 1 to 63. When the difference becomes one for the first time, the corresponding OffsetNumberBits value is determined to be that of the $LogFile.

Another method can be used when there is no circulation in the normal logging area. This method is based on the fact that the position of the record according to the starting position of the $LogFile can be obtained by multiplying the offset number obtained from the This LSN value of the record by eight. The OffsetNumberBits value is adjusted according to the size of the $LogFile. And the record offset, which is eight times the maximum offset number that can be expressed with the OffsetNumberBits value, must be at least equal to or greater than the file size in order to express the position of the last record at the end of the file (see Fig. 9).

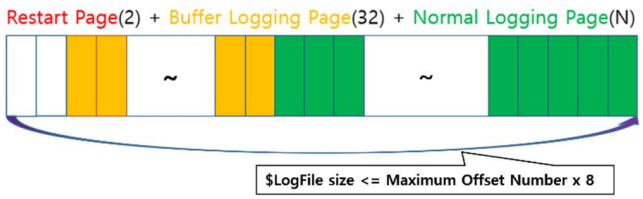



**FIGURE 9.** Relationship between the size of the $LogFile and the offset number.

The method used to obtain the OffsetNumberBits value is as follows: The estimated size of the $LogFile is first obtained by adding $0 \times 3000$, which is the sum of the sizes of the restart area and the last logging page, to the difference between the location of the first and the last carved logging pages (see Fig. 10).

Then, the maximum offset number is obtained by dividing the estimated size of the $LogFile by eight, and its value is changed to a binary number. The position of the highest 1 in the binary number is identified. The value of OffsetNumberBits represents the distance between the bottom and the position of the highest 1. Algorithm 2 is a pseudocode for

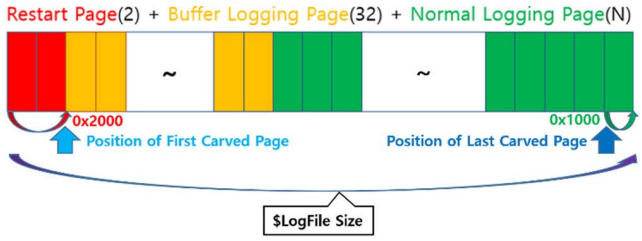



**FIGURE 10.** Calculating the size of the $LogFile.

finding the OffsetNumberBits value of the $LogFile when the restart page is not carved.

**Algorithm 2** Find the OffsetNumberBits of $LogFile

```
if there is a rotation in normal logging area then
    last_lsn_1 ← the smallest last_lsn in normal logging area
    last_lsn_2 ← the biggest last_lsn in normal logging area
    find_flag ← FALSE
    for i=1;i<64;i++ do
        seq_num_1 ← last_lsn_1 >> i
        seq_num_2 ← last_lsn_2 >> i
        if seq_num_2 - seq_num_1 == 1 then
            find_flag ← TRUE
            offset_number_bits ← i
            break
        end if
    end for
    if find_flag == FALSE then
        offset_number_bits ← 24
    end if
else
    file_offset_1 ← file offset of first carved logging page
    file_offset_2 ← file offset of last carved logging page
    file_size ← (file_offset_2 - file_offset_1) + 0x3000
    maximum_offset_number ← file_size / 8
    binary_arr[64] ← ConvertUint64ToBinaryArray(maximum_offset_number)
    for i=63;i»0;i- do
        if binary_arr[i] == 1 then
            offset_number_bits ← i
            break
        end if
    end for
end if
```

If the OffsetNumberBits value cannot be obtained through the two methods described above, it is assigned the value 24, which corresponds to the most common size of the $LogFile (64 MB).

### *E. STEP 3: RECOVERING UNCARVED LOGGING PAGES*

Once the structure of the carved logging pages has been identified, logging pages that have not been carved due to damage to the page header should be recovered. This is because the $LogFile data are likely to persist even in logging pages that have not been carved.

#### 1) IN CASE RESTART PAGE IS CARVED

Once the restart page has been carved, the ranges of the buffer logging area and the normal logging area can be accurately identified through Table 3. Uncarved pages for each logging area are recovered as follows:

In case of the normal logging area, the sequence number is first collected through the Last LSN in the header of the carved logging pages. The page that has not been carved within the area is then searched for. If an uncarved page is found, a record carving signature is created based on the sequence number collected earlier. The generated signature is then used to search forward in units of eight bytes from the end of the page to find the last record, including records that go beyond the page. If the last record is found, its This LSN value is the Last LSN value of the uncarved page. Algorithm 3 is a pseudocode used to obtain the Last LSN value by searching for the last record in the page.

**Algorithm 3** Obtaining the Last LSN From Uncarved Pages

```
function CheckPageForLastLSN(current_file_offset, seq_num[])
    page_buffer[] ← GetPageData(current_file_offset)
    current_position ← 0xFC8
    detection_position ← -1
    find_flag ← FALSE
    page_seq_num ← -1
    while current_position >= 0 do
        record_header ← page_buffer[current_position]
        this_lsn_seq_num ← record_header.this_lsn >> offset_num_bits
        for seq_num in seq_num[] do
            if this_lsn_seq_num == seq_num then
                pre_lsn_seq_num ← record_header.previous_lsn >> offset_num_bits
                if pre_lsn_seq_num == seq_num
                  || record_header.previous_lsn == 0 then
                    undo_lsn_seq_num ← record_header.undo_lsn >> offset_num_bits
                    if undo_lsn_seq_num == seq_num
                      || record_header.undo_lsn == 0 then
                        if record_header.record_type == 0x1
                          || record_header.record_type == 0x2 then
                            if record_header[0x31] == 0x0
                              || record_header[0x33] == 0x0 then
                                detection_position ← current_position
                                find_flag ← TRUE
                                page_seq_num ← seq_num
                                break
                            end if
                        end if
                    end if
                end if
            end if
        end for
        if find_flag == TRUE then
            break
        end if
        current_position ← current_position - 0x8
    end while
    if detection_position >= 0 then
        record_header ← page_buffer[detection_position]
        next_record_position ← record_header.client_data_length + detection_position
                                + 0x30
        if next_record_position >= 0x1000 then
            last_lsn ← record_header.this_lsn
            return last_lsn
        else
            record_header ← page_buffer[next_record_position]
            this_lsn_seq_num ← record_header.this_lsn >> offset_num_bits
            if this_lsn_seq_num == page_seq_num then
                last_lsn ← record_header.this_lsn
                return last_lsn
            else
                record_header ← page_buffer[detection_position]
                last_lsn ← record_header.this_lsn
                return last_lsn
            end if
        end if
    end if
end function
```

When the Last LSN value is obtained in this way, a virtual header is created and Last LSN is set to recover the damaged

header of the uncarved page. If the last record is not found and the last LSN value is not obtained, and the virtual header uses the last LSN value of the page located directly in front of it. Normal logging pages recovered by creating a virtual header in this process are added to the list of normal logging pages.

The recovery of uncarved logging pages in the buffer logging area depends on the version of LFS. If the version of LFS is 1.1, the Last End LSN value is used instead of the Last LSN value because the buffer logging page does not store the Last LSN value in Last LSN field of the header. It instead stores the file offset value that represents the location in the $LogFile where the data of the buffer page are applied later. The file offset value is not related to records in the page and is not used for record carving in the subsequent process. Therefore, if an uncarved page is found in the buffer logging area, the record carving signature is used to search forward in units of eight bytes from the end of the page to find the last record among those that do not go beyond the page. If this record is found, its This LSN value is the Last End LSN value of the uncarved page. The record carving signature used in this process is generated using the sequence number obtained from the carved logging pages as described above. Algorithm 4 is a pseudocode for obtaining the Last End LSN value by searching for the last record in the page.

**Algorithm 4** Obtaining the Last End LSN in Uncarved Pages

```
function CheckPageForLastEndLSN(current_file_offset, seq_num[])
    page_buffer[] ← GetPageData(current_file_offset)
    current_position ← 0xFC8
    detection_position ← -1
    while current_position >= 0 do
        record_header ← page_buffer[current_position]
        this_lsn_seq_num ← record_header.this_lsn >> offset_num_bits
        for seq_num in seq_num[] do
            if this_lsn_seq_num == seq_num then
                pre_lsn_seq_num ← record_header.previous_lsn >> offset_num_bits
                if pre_lsn_seq_num == seq_num
                  || record_header.previous_lsn == 0 then
                    undo_lsn_seq_num ← record_header.undo_lsn >> offset_num_bits
                    if undo_lsn_seq_num == seq_num
                      || record_header.undo_lsn == 0 then
                        if record_header.record_type == 0x1
                          || record_header.record_type == 0x2 then
                            if record_header[0x31] == 0x0
                              || record_header[0x33] == 0x0 then
                                if current_position + 0x30
+record_header.client_data_length
                                < 0x1000 then
                                last_end_lsn ← current_position
                                return last_end_lsn
                                end if
                            end if
                        end if
                    end if
                end if
            end if
        end for
        current_position ← current_position - 0x8
    end while
end function
```

Once the Last End LSN value has been obtained in this way, a virtual header is created and the Last End LSN is set to recover the damaged header of the uncarved page.

In case of LFS 2.0, virtual headers of the uncarved logging pages are created in the same way as in the normal logging area to execute recovery because the Last LSN value is used in the header of the buffer logging page. As such, the recovered buffer logging pages in each LFS version are added to the list of buffer logging pages.

#### 2) IN CASE NO RESTART PAGE IS CARVED

If the restart page is not carved, the exact range of buffer logging and normal logging areas is unknown because the size of the $LogFile cannot be determined. Accordingly, the uncarved page can be recovered only in the area between the first and last of the carved logging pages. When the area in which the recovery operation is to be performed is designated in this way, the subsequent recovery process is the same as when the restart page is carved as described above.

### *F. STEP 4: SORTING LOGGING PAGES*

Once the uncarved logging pages have been recovered, and information on the buffer and normal logging pages has been collected, the logging pages should be sorted in order for the following reasons: In some cases, the $LogFile record is located between logging pages, or may use multiple logging pages owing to its large size. The logging pages should be analyzed in order in this case to acquire the data on such $LogFile records. When the storage device formatted using NTFS is unmounted and remounted, the structure of circulation of the normal logging area of $LogFile may be broken and the order of logging pages may become mixed. The logging pages should be analyzed in order in this case to obtain as much data as possible (see Fig. 11).

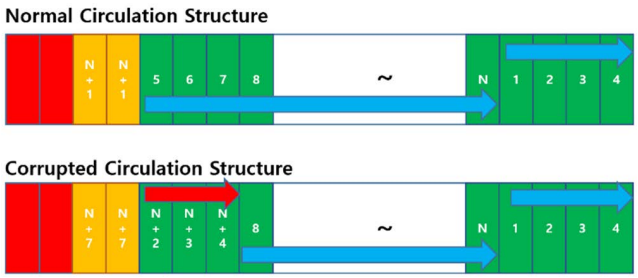



**FIGURE 11.** Normal circulation vs. corrupted circulation.

The method for sorting logging pages in order is as follows: While sequentially searching for logging pages in the normal logging area, pages whose Last LSN values increase or remain the same are grouped together. Then, each page group is sorted in order based on the largest Last LSN value in the group. Finally, the buffer logging pages are sorted according to the version of LFS. The largest Last LSN value of the normal logging area is used as a reference value for sorting (see Fig. 12).

In case of LFS 1.1, the Last End LSN value is used because the Last LSN values of the two buffer logging pages cannot be used. If the Last End LSN values of

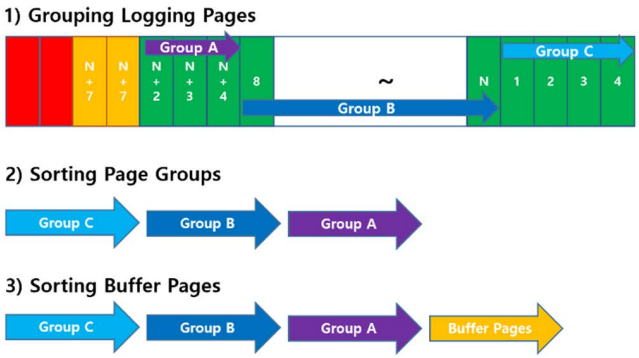



**FIGURE 12.** Sorting the logging pages.

both buffer logging pages are less than the reference value, it means that the data of both buffer logging pages are already reflected in the normal logging area. Therefore, the two buffer pages are not sorted. If the Last End LSN value of one of the two buffer logging pages is greater than the reference value, only the corresponding buffer logging page is sorted. Finally, if the Last End LSN of both buffer logging pages are greater than the reference value, only the buffer logging page with the larger Last End LSN value is sorted.

In case of LFS 2.0, the buffer logging pages with Last LSN value smaller than the reference value are excluded. The remaining pages are sorted based on their Last LSN values and grouped together. Finally, the group of buffer logging pages is sorted.

### G. STEP 5: CARVING $LogFile RECORDS

Once the logging pages have been sorted, record carving is performed on them to obtain as much of the data on the record that remains on the disk as possible, even if data on the logging page have been damaged by an accident/disaster or cyber terrorism.

Record carving is performed in units of logging pages, and a record carving signature that is valid only within a given page is generated using the sequence number obtained through the Last LSN value of the page header. The record is searched for by using the signature in units of eight bytes on data on the page (see Fig. 13).

If the header and data of a carved record continue to the next page, a record carving signature that is valid only on next page is regenerated and the record carving operation is resumed after obtaining the remaining header and data from the next page.

The $LogFile data acquired through record carving in this way can be used to create file-level events, such as file creation, deletion, name change, and move [17], or to track the data history of a specific file [19].

Figure 14 shows a flowchart summarizing the process of $LogFile data recovery through the record carving technique described above.

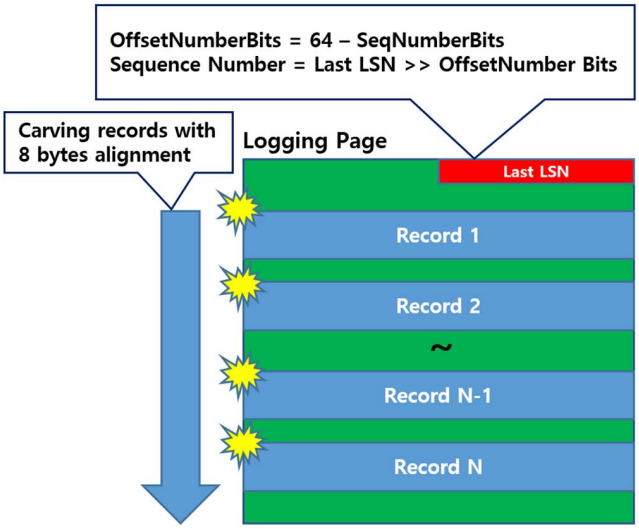



**FIGURE 13.** Carving $LogFile records.

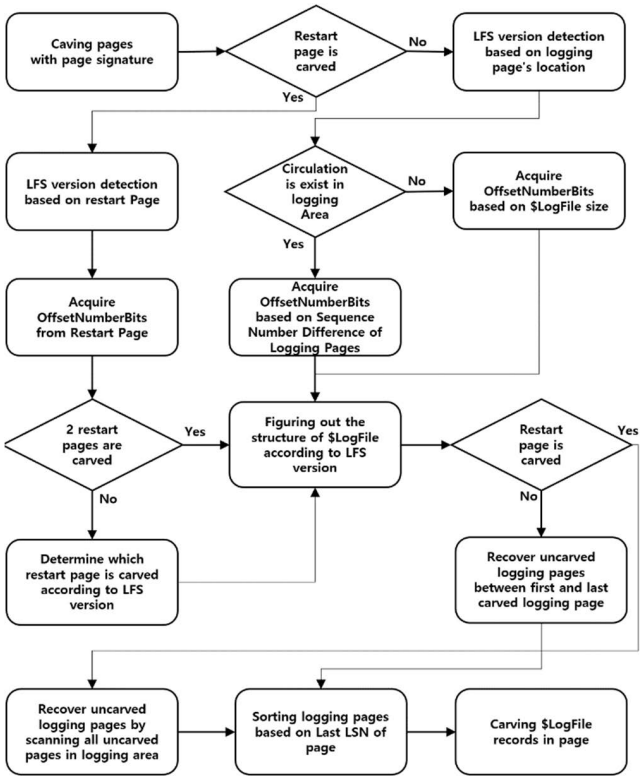



**FIGURE 14.** Flowchart for the recovery of $LogFile data.

## VI. EXPERIMENT

This section describes the development of the record carving-based $LogFile data recovery technology based on the above content, and evaluates its performance in comparison with currently existing tools through experiments.

### A. TOOL DEVELOPMENT

The technology for $LogFile data recovery based on record carving was developed in modular format. The module

receives a disk or volume image file as input, and each of its sub-modules operates to finally output the $LogFile record data. (see Fig. 15).

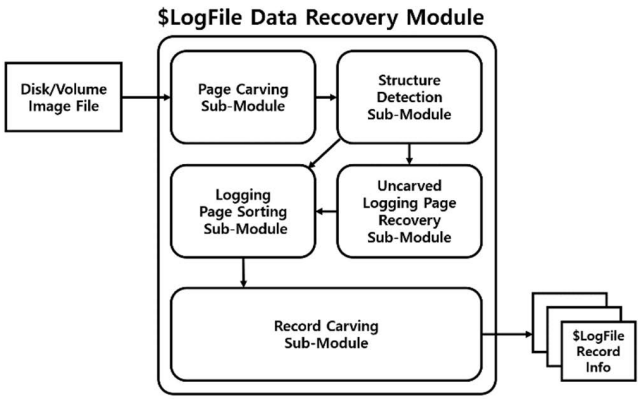



**FIGURE 15.** **$LogFile data recovery module.**

The developed module was added to the NTFS Log Tracker v1.8 [17] and the NTFS Data Tracker v1.1 [19]. To use the functions of the module in each tool, the path of the disk/volume image file needs be input to the path of the source file for carving as shown in Figure 16.

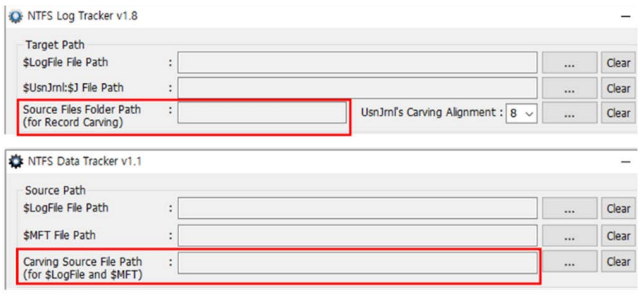



**FIGURE 16.** **Input interface of disk/volume image file in each tool.**

Each tool uses data on the record received from the $LogFile data recovery module for file event analysis, and to analyze the file data history.

The NTFS Log Tracker and NTFS Data Tracker with the record carving-based $LogFile data recovery module can be downloaded free of charge.[1]

### *B. EVALUATION*

We compared the performance to recover $LogFile data between NTFS Log Tracker and NTFS Data Tracker which use the module of $LogFile data recovery developed in this paper, and existing $LogFile recovery tools such as Bulk Extractor v2.0, X-Ways Forensics v19.9 SR-4.

#### 1) DATA SET

The test image files used for performance evaluation were created by storing data of the $LogFile (64 MB) to which the damage scenario is applied in the middle of an image file (1 GB) to fit the sector layout (512 bytes).

The scenarios of damage to the $LogFile data considered were as follows: The first was the case where the $LogFile data had not been corrupted at all. We tested whether the $LogFile data could be recovered in case the $LogFile could not be accessed normally due to damage to the file system. The test image file (test01.dd) created in this scenario was used as a baseline dataset for performance evaluation because all $LogFile data within the file could be recovered by the existing recovery methods and proposed recovery method in this paper. Therefore, the existing recovery methods and proposed recovery method could be confirmed whether $LogFile data is recovered properly while adding damage scenarios based on this image file. The second was the case where the header of the restart page had been damaged, and was divided into the case where the header of the first restart page had been damaged for each LFS version, one where the header of the second restart page had been damaged for each LFS version, and the case where the headers of all restart pages had been damaged. These scenarios were run to determine whether different structures of the $LogFiles for each version of the LFS could be identified and the $LogFile data could be recovered. The third case considered was one where the header of the buffer logging page had been damaged. In this case, 50% of page headers in the buffer logging area for each version of the LFS were damaged. These scenarios were run to determine whether buffer logging pages with damaged headers could be recovered according to the characteristics of different buffer logging pages for each LFS version, and whether the $LogFile data could be recovered. The fourth scenario was one where the header of normal logging page had been damaged. In this case, because there was no difference between the LFS versions, 50% of the page headers in the normal logging area were assumed to have been damaged only for LFS 2.0. This scenario was run to determine whether the $LogFile data could be restored by recovering the pages with damaged headers in the normal logging area. The fifth was the case where data on the normal logging pages had been damaged. In this case, there was no difference between the LFS versions, and thus only LFS 2.0 was considered. One record within data on pages corresponding to 50% of the normal logging area was selected and damaged. This scenario was run to determine whether the $LogFile data could be recovered by carving records in the remaining area, except for the damaged area. The last scenario was one where the header and data of pages in the normal logging area had been simultaneously damaged, and the fourth and fifth scenarios were applied at the same time. This scenario was run to determine whether the logging pages with damaged headers could be recovered, and whether records in the areas except the damaged areas could be carved to recover the $LogFile data.

Twelve image files were used for the evaluation, and the LFS version, damage-related scenario, and details of each image are shown in Table 4. In each scenario, the header was

[1] URL: https://sites.google.com/site/forensicnote/ntfs-log-tracker, https://sites.google.com/site/forensicnote/ntfs-data-tracker

**TABLE 4.** Damage to and purpose of testing each image file.

| Image File | LFS Version | Damage Scenarios | Test Details |
|---|---|---|---|
| test01.dd | 1.1 | Nothing | Check if all data are acquired from the undamaged $LogFile |
| test02.dd | 1.1 | Corrupt header of first restart page | When the header of the first restart page is damaged in LFS 1.1, check if the structure of the $LogFile is properly identified and analyzed. |
| test03.dd | 1.1 | Corrupt header of second restart page | When the header of the second restart page is damaged in LFS 1.1, check if the structure of the $LogFile is properly identified and analyzed. |
| test04.dd | 1.1 | Corrupt header of all restart pages | When the headers of all restart pages are damaged in LFS 1.1, check if the structure of the $LogFile is properly identified and analyzed. |
| test05.dd | 2.0 | Corrupt header of first restart page | When the header of the first restart page is damaged in LFS 2.0, check if the structure of the $LogFile is properly identified and analyzed. |
| test06.dd | 2.0 | Corrupt header of second restart page | When the header of the second restart page is damaged in LFS 2.0, check if the structure of the $LogFile is properly identified and analyzed. |
| test07.dd | 2.0 | Corrupt header of all restart pages | When the headers of all restart page are damaged in LFS 2.0, check if the structure of the $LogFile is properly identified and analyzed. |
| test08.dd | 1.1 | Corrupt header of first buffer logging page | When the header of the first buffer logging page is damaged in LFS 1.1, check if the damaged logging page is normally analyzed. |
| test09.dd | 2.0 | Corrupt headers of 50% of pages in buffer logging area | When the headers of many logging pages in the buffer logging area are damaged in LFS 2.0, check if they are normally analyzed. |
| test10.dd | 2.0 | Corrupt headers of 50% of pages in normal logging area | When the headers of many logging pages in the normal logging area are damaged, check if they are normally analyzed. |
| test11.dd | 2.0 | Corrupt one record in each of 50% pages in normal logging area (zeroize 4,359 records) | When a number of records in the normal logging pages are damaged, check if all records are normally analyzed except for the damaged ones. |
| test12.dd | 2.0 | Corrupt header and one record in each of 50% pages in normal logging area (zeroize 4,305 records) | After processing the logging pages with damaged headers in the normal logging area, check if all records are normally analyzed except for the damaged ones. |

corrupted by zeroizing the initial 32-byte data, including the magic number, and the record was corrupted by zeroizing the header and data of the record. In case of test11.dd and test12.dd image files, one record that does not go over the page is selected in each of 50% pages in normal logging area, and it is zeroized. However, depending on the page, there were cases where there was no record header, and only record data, in the page. The actual number of zero-filled records was thus 4,359 in the test11.dd image file and 4,305 in the test12.dd image file.

### 2) RESULT

The performance of each tool was assessed differently because the results of recovery of each were different. In case of X-Ways Forensics v19.9 SR-4, which performs file carvi ng, the hash values of the carved file from the test image file were compared with those of the $LogFile applied in the damage scenarios to check whether the file had been carved normally. In case of Bulk Extractor v2.0 that performs page carving, the number of pages carved from the test image file is compared with those in the original $LogFile to check whether all pages had been carved normally. Finally, in case of the NTFS Log Tracker and NTFS Data Tracker, the number of records that the tools analyzed in the test image file was compared with the number of records remaining in the $LogFile, to which the damage scenario had been applied, to check whether all records to be analyzed had in fact been carved.

Table 5 shows the results of the performance evaluation of each tool on the test image files. In the results, "O" means that all $LogFile data that could be obtained from the test image were successfully acquired, and "" means that all data were not acquired but those from the logging area were. Finally, "X" means that the data of the logging area were not properly obtained owing to structural damage.

X-Ways Forensics v19.9 SR-4 successfully performed file carving on the test01.dd image file, which had sustained no damage, but if the header of the restart page or the logging page had been damaged, it was unable to carve the file norm ally. However, it successfully performed file carving on the test11.dd image file which only records were damaged.

Bulk Extractor v2.0 was able to carve all pages of the test01.dd image file that had sustained no damage. In case the header of the restart page had been damaged (test02.dd∼test07.dd), the carving of all logging pages except for the restart page was successful. In addition, if only records had been damaged, all logging pages were carved well. However, if the header of the logging page had been damaged, the corresponding page was not carved properly.

**TABLE 5. Results of tool evaluation.**

| Image File | X-Ways Forensics v19.9 SR-4 (MD5 hash of carved $LogFile file / MD5 hash of $LogFile file to which the damage scenario is applied ) | Bulk Extractor v2.0 (Carving Rate of Restart Page, Carving Rate of Logging Page) | NTFS Log Tracker / NTFS Data Tracker (Number of analyzed records / Number of existed records in $LogFile) |
|---|---|---|---|
| test01.dd | O (e900c6fb0e49be88d2d123bae8489841 / e900c6fb0e49be88d2d123bae8489841) | O (2/2, 16,382/16,382) | O (282,665/282,665) |
| test02.dd | X (File caving failed) | △ (1/2, 16,382/16,382) | O (282,665/282,665) |
| test03.dd | X (File caving failed) | △ (1/2, 16,382/16,382) | O (282,665/282,665) |
| test04.dd | X (File caving failed) | △ (0/2, 16,382/16,382) | O (282,665/282,665) |
| test05.dd | X (File caving failed) | △ (1/2, 16,382/16,382) | O (253,124/253,124) |
| test06.dd | X (File caving failed) | △ (1/2, 16,382/16,382) | O (253,124/253,124) |
| test07.dd | X (File caving failed) | △ (0/2, 16,382/16,382) | O (253,124/253,124) |
| test08.dd | X (File caving failed) | X (2/2, 16,381/16,382) | O (282,665/282,665) |
| test09.dd | X (File caving failed) | X (2/2, 16,366/16,382) | O (253,124/253,124) |
| test10.dd | X (3e96b0774ae01ba1284ccb4da86aeab8 /9bd5ff0e492940a6f2fa55017925c53b | X (2/2, 8,207/16,382) | O (253,124/253,124) |
| test11.dd | O (6d839f2a035db51a5d62256a460d1124 /6d839f2a035db51a5d62256a460d1124 | O (2/2, 16,382/16,382) | O (248,765/248,765) |
| test12.dd | X (ba2142879c7b67eed609530aa956b0f9 /2dcbe74c5e45f19a198b6405eafd50c2) | X (2/2, 8,207/16,382) | O (248,819/248,819) |

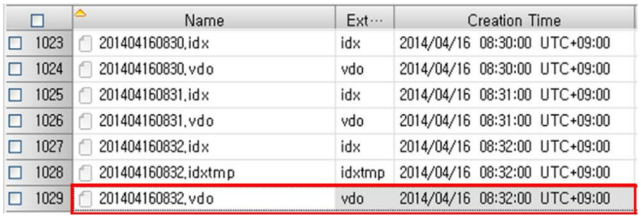
The NTFS Log Tracker and NTFS Data Tracker analyzed all records normally, even if the header of the restart page or the logging page had been damaged. They were also able to normally analyze all records in test11.dd and test12.dd except for the damaged ones.

The above results show that the proposed methods of recovery based on record carving were able to obtain as much $LogFile data as possible, even when the structure of the LogFile had been damaged such that existing tools did not work. The test images used in the performance evaluation can be downloaded from the URL where the tool can be downloaded.

## VII. CASE STUDY

At around 8:34 am on April 16, 2014, the ferry MV Sewol sank off the coast of Jindo in South Korea [8]. The ferry was equipped with a Windows XP-based CCTV system that stored one-minute CCTV video files (.vdo) on the F drive. After the accident, no CCTV videos could be obtained at 8:33 am and 8:34 am, corresponding to the time at which the ferry sank. It then became an important to determine whether the CCTV files had been intentionally deleted or had not existed in the first place.

The F drive of the CCTV system was formatted with NTFS. When the metadata of the file stored in $MFT of the NTFS



| | Name | Ext··· | Creation Time |
|---|---|---|---|
| 1023 | 201404160830.idx | idx | 2014/04/16 08:30:00 UTC+09:00 |
| 1024 | 201404160830.vdo | vdo | 2014/04/16 08:30:00 UTC+09:00 |
| 1025 | 201404160831.idx | idx | 2014/04/16 08:31:00 UTC+09:00 |
| 1026 | 201404160831.vdo | vdo | 2014/04/16 08:31:00 UTC+09:00 |
| 1027 | 201404160832.idx | idx | 2014/04/16 08:32:00 UTC+09:00 |
| 1028 | 201404160832.idxtmp | idxtmp | 2014/04/16 08:32:00 UTC+09:00 |
| 1029 | 201404160832.vdo | vdo | 2014/04/16 08:32:00 UTC+09:00 |

**FIGURE 17. Results of analyzing $MFT.**

were analyzed, only traces of the 201404160832.vdo file, which was saved at 8:32 am, remained as shown in Figure 17.

However, when the $LogFile data were analyzed through the NTFS Data Tracker, traces of the 201404160833.vdo and 201404160834.vdo files, corresponding to 8:33 and 8:34 am, were found as shown in Figure 18.

When the data history of 201404160832.vdo was analyzed through the NTFS Data Tracker, the 80th data runs matched data runs of the 201404160832.vdo file recorded in $MFT, as shown in Figures 19 and 20.

If the 201404160833.vdo, 201404160834.vdo files had been normally stored and then deleted, data runs of 201404160832.vdo recorded in the $MFT would have matched the last data runs of the history of the file as analyzed through the $LogFile. However, data runs of

File List

| No | Unknown | Grouping | FileName | FilePath |
|---|---|---|---|---|
| 1123 | | | 201404160832.vdo | ₩20140416₩201404160832.vdo |
| 1124 | | | 201404160832.idx | ₩20140416₩201404160832.idx |
| 1125 | | Y | 201404160832.idxtmp | ₩20140416₩201404160832.idxtmp |
| 1126 | | | 201404160833.vdo | ₩20140416₩201404160833.vdo |
| 1127 | | | 201404160833.idx | ₩20140416₩201404160833.idx |
| 1128 | | Y | 201404160833.idxtmp | ₩20140416₩201404160833.idxtmp |
| 1129 | | | 201404160834.vdo | ₩20140416₩201404160834.vdo |
| 1130 | | | 201404160834.idx | ₩20140416₩201404160834.idx |
| 1131 | | Y | 201404160834.idxtmp | ₩20140416₩201404160834.idxtmp |

**FIGURE 18.** Result of analyzing $LogFile with NTFS Data Tracker.

| | Start Sector | End Sector | Start Cluster | Clusters |
|---|---|---|---|---|
| 1 | 585,279,608 | 585,279,719 | 73,159,951 | 14 |
| 2 | 597,797,416 | 597,860,591 | 74,724,677 | 7,897 |
| 3 | 603,804,912 | 603,804,983 | 75,475,614 | 9 |
| 4 | 629,300,184 | 629,335,895 | 78,662,523 | 4,464 |

**FIGURE 19.** Data runs of 201404160832.vdo from $MFT.

File List

| No | Unknown | Grouping | FileName |
|---|---|---|---|
| 1116 | | | 201404160831.idx |
| 1117 | | Y | 201404160831.idxtmp |
| 1118 | | | 201404160832.vdo |
| 1119 | | | 201404160832.idx |
| 1120 | | Y | 201404160832.idxtmp |

Data History $DATA(Default)

| Order | EventInfo | DataInfo |
|---|---|---|
| 77 | Update DataRuns | DataRuns : 73159951(14), 74724677(7897), 75475614(9), 78662523(4048) |
| 78 | Update DataRuns | DataRuns : 73159951(14), 74724677(7897), 75475614(9), 78662523(4112) |
| 79 | Update DataRuns | DataRuns : 73159951(14), 74724677(7897), 75475614(9), 78662523(4128) |
| 80 | Update DataRuns | DataRuns : 73159951(14), 74724677(7897), 75475614(9), 78662523(4464) |
| 81 | Update DataRuns | DataRuns : 73159951(14), 74724677(7897), 75475614(9), 78662523(4752) |
| 82 | Update DataRuns | DataRuns : 73159951(14), 74724677(7897), 75475614(9), 78662523(5232) |
| 83 | Update DataRuns | DataRuns : 73159951(14), 74724677(7897), 75475614(9), 78662523(5296) |

**FIGURE 20.** File data history of 201404160832.vdo from $LogFile.

201404160832.vdo in the current $MFT matched the 80th data runs in the history of the file. These traces prove that only up to the 80th data runs in the data history of the 201404160832.vdo file had been saved to $MFT in the disk; then, as the system suddenly stopped, the last data runs of 201404160832.vdo in the memory were not saved to disk. It also proves that the metadata of 201404160833.vdo and 201404160834.vdo files were not saved to $MFT in the disk. Therefore, there were no traces of 201404160833.vdo and 201404160834.vdo not because the corresponding video files were deleted, but because their metadata were not saved to $MFT due to the sudden termination of the system.

## VIII. CONCLUSION

A file system is the basic structure that most operating systems use to store files, and is the most basic and important element for investigators to analyze during digital forensic investigations. The recovery of file system metadata enables investigations to proceed even when the file system structure is damaged and files cannot be accessed normally, and can help to find additional evidence in unallocated areas.

**TABLE 6.** Structure of the header of a record.

| Field | Offset (Size) | Detail |
|---|---|---|
| This LSN | +0x0 (8) | The LSN of the current record |
| Previous LSN | +0x8 (8) | The LSN of the previous record |
| Client Undo LSN | +0x10 (8) | The LSN of the next Undo operation |
| Client Data Length | +0x18 (4) | The size of the record |
| Client ID | +0x1C (4) | The owner ID of the current record. |
| Record Type | +0x20 (4) | 0x02 (Checkpoint Record), 0x01 (The others) |
| Transaction ID | +0x24 (4) | The value used by Transaction Manager |
| Flags | +0x28 (2) | 0x01 (the current record goes over the page), 0x00 (the current record does not go over the page) |
| Redo OP | +0x30 (2) | The type of redo operation |
| Undo OP | +0x32 (2) | The type of undo operation |
| Redo Offset | +0x34 (2) | The offset of Redo Data |
| Redo Length | +0x36 (2) | The size of Redo Data |
| Undo Offset | +0x38 (2) | The offset of Undo Data |
| Undo Length | +0x3A (2) | The size of Undo Data |
| LCN to Follows | +0x3E (2) | 0x01(There is next record), 0x00(There is no next record) |
| Record Offset | +0x40 (2) | The offset of the attribute to which the operation is applied in the MFT entry |
| Attr Offset | +0x42 (2) | The offset of the position to which the operation is applied in the attribute |
| MFT Cluster Index | +0x44 (2) | The value for the position of MFT entry targeted by operation in the cluster pointed by target LCN/VCN<br>1st (0x0000), 2nd (0x0002), 3rd (0x0004), 4th (0x0006) |
| Target VCN | +0x48 (4) | A sequential number of the cluster which has MFT entry to which the operation is applied in the volume |
| Target LCN | +0x50 (4) | A sequential number of the cluster which has MFT entry to which the operation is applied in the non-resident stream of $MFT |

In this paper, we studied the recovery of $LogFile records, the metadata of NTFS, one of the most used file systems worldwide. We identified the mechanisms of record storage through a detailed analysis of the $LogFile structure and proposed a recovery algorithm for records without a fixed value. The proposed algorithm is applicable to all versions of $LogFile and has been released as a freeware tool. In addition, we confirmed the superiority of the proposed algorithm via experiments the evaluated performance against existing $LogFile recovery tools. Finally, we demonstrated the effectiveness of this study with a real case solved by the recovery

of $LogFile data in a situation where the file system was damaged due to an accident/disaster.

The results of this study should be useful for digital forensic investigators in recovering $LogFile data at the time of system shutdown in the case of an accident/disaster or a cyber terrorism so that they can analyze the causes of the event, reconstruct it, and devise methods to prevent it in the future.

Journal data, one of file system metadata, contains important information that can be used to accurately identify events that occurred just before the system was stopped. In particular, the recovery of journal data is a very important task in situations where the file system is damaged by accidents/disasters or cyber terrorism. In future studies, we plan to study regarding forensic recovery of journal data in Ext4, which is the most used file system along with NTFS.

## APPENDIX

See Table 6.

## REFERENCES

[1] B. Carrier, *File System Forensic Analysis*. Boston, MA, USA: Addison-Wesley Professional, 2005.
[2] G.-S. Cho, "NTFS directory index analysis for computer forensics," in *Proc. 9th Int. Conf. Innov. Mobile Internet Services Ubiquitous Comput.*, Santa Cantarina, Brazil, Jul. 2015, pp. 441–446.
[3] K. H. Hansen and F. Toolan, "Decoding the APFS file system," *Digit. Invest.*, vol. 22, pp. 107–132, Sep. 2017.
[4] K. D. Fairbanks, "An analysis of Ext4 for digital forensics," *Digit. Invest.*, vol. 9, pp. 118–130, Aug. 2012.
[5] X. Lin, "Deleted file recovery in NTFS," in *Introductory Computer Forensics*. Cham, Switzerland: Springer, Nov. 2018, pp. 199–210.
[6] J. Plum and A. Dewald, "Forensic APFS file recovery," in *Proc. 13th Int. Conf. Availability, Rel. and Secur.*, Hamburg, Germany, 2018, pp. 1–10.
[7] D. Kim, J. Park, K. G. Lee, and S. Lee, "Forensic analysis of Android phone using ext4 file system journal log," in *Future Information Technology, Application, and Service* (Lecture Notes in Electrical Engineering), vol. 1. Berlin, Germany: Springer, Jun. 2012, pp. 435–446.
[8] *Sinking of MV Sewol*, Accessed: May 26, 2022. [Online]. Available: https://en.wikipedia.org/wiki/Sinking_of_MV_Sewol
[9] *2013 (South) Korea Cyberattack*, Accessed: May 26, 2022. [Online]. Available: https://en.wikipedia.org/wiki/2013_South_Korea_cyberatt ack
[10] J. Oh. (2013). *Advanced $UsnJrnl Forensics*. Accessed: May 26, 2022. [Online]. Available: http://forensicinsight.org/wp-content/uploads/2013/07/F-INSIGHT-Advanced-UsnJrnl-Forensics-English.pdf
[11] M. Fuchs. (2018). *MFTEntryCarver*. Accessed: May 26, 2022. [Online]. Available: https://github.com/cyb3rfox/MFTEntryCarver
[12] H. Segev. (2021). *INDXRipper*. Accessed: May 26, 2022. [Online]. Available: https://github.com/harelsegev/INDXRipper
[13] LSoft Technologies. *APFS Recovery Methodologies*. Accessed: May 26, 2022. [Online]. Available: https://www.ntfs.com/apfs-recovery.htm
[14] A. Dewald and S. Seufert, "AFEIC: Advanced forensic Ext4 inode carving," *Digit. Invest.*, vol. 20, pp. 83–91, Mar. 2017.
[15] *X-Ways Forensics*. Accessed: May 26, 2022. [Online]. Available: http://www.x-ways.net/winhex/manual.pdf
[16] S. Garfinkel. (2012). *BulkExtractor*. Accessed: May 26, 2022. [Online]. Available: https://github.com/simsong/bulk_extractor
[17] J. Oh. (2013). *NTFS Log Tracker*. Accessed: May 26, 2022. [Online]. Available: http://forensicinsight.org/wp-content/uploads/2013/06/F-INSIGHT-NTFS-Log-TrackerEnglish.pdf
[18] D. Cowen. (2013). *NTFS Triforce—A Deeper Look Inside the Artifacts*. Accessed: May 26, 2022. [Online]. Available: https://www.hecfblog.com/2013/01/ntfs-triforce-deeper-look-inside.hl
[19] J. Oh, S. Lee, and H. Hwang, "NTFS data tracker: Tracking file data history based on $LogFile," *Digit. Invest.*, vol. 39, Dec. 2021, Art. no. 301309.
[20] R. Nordvik, K. Porter, F. Toolan, S. Axelsson, and K. Franke, "Generic metadata time carving," *Forensic Sci. Int., Digit. Invest.*, vol. 33, Jul. 2020, Art. no. 301005.
[21] K. Porter, R. Nordvik, F. Toolan, and S. Axelsson, "Timestamp prefix carving for filesystem metadata extraction," *Forensic Sci. Int., Digit. Invest.*, vol. 38, Sep. 2021, Art. no. 301266.
[22] M. Russinovich, D. Solomon, and A. Lonescu, *Windows Internals, Part 2*, 6th ed. Unterschleissheim, Germany: Microsoft Press, 2012.
[23] *NTFS Documentation-File-$LogFile(2)*. Accessed: May 26, 2022. [Online]. Available: https://flatcap.github.io/linux-ntfs/ntfs/files/logfile.html
[24] *Cyberterrorism*, Accessed: May 26, 2022. [Online]. Available: https://en.wikipedia.org/wiki/Cyberterrorism
[25] M. D. Martin. *Tracing the Lineage of DarkSeoul*. Accessed: May 26, 2022. [Online]. Available: https://www.giac.org/paper/gsec/31524/tracing-lineage-darkseoul/126346
[26] J. Kim. *3.20 Cyber-Terror Form Recovery Perspectives*. Accessed: May 26, 2022. [Online]. Available: http://forensic-proof.com/archives/5111
[27] Msuhanov. *How the $LogFile Works?*. Accessed: May 26, 2022. [Online]. Available: https://dfir.ru/2019/02/16/how-the-logfile-works

**JUNGHOON OH** received the B.S. degree from the Division of Computer, Information Communication Engineering, Dongguk University, in 2010. He is currently pursuing the Ph.D. degree with the Graduate School of Information Security, Korea University. His research interests include digital forensics, filesystem forensics, incident response, and artificial intelligence.

**SANGJIN LEE** received the Ph.D. degree from the Department of Mathematics, Korea University, in 1994. From 1989 to 1999, he was a Senior Researcher at the Electronics and Telecommunications Research Institute, South Korea. He has been running the Digital Forensic Research Center, Korea University, since 2008, where he is currently the President of the Division of Information Security. He has authored or coauthored over 130 papers in various archival journals and conference proceedings and over 200 articles in domestic journals. His research interests include digital forensics, data processing, forensic framework, and incident response.

**HYUNUK HWANG** (Member, IEEE) received the Ph.D. degree from the Department of Information Security, Chonnam National University, in 2004. He is currently the Head of Department of the Attached Institute of ETRI. His research interests include digital forensics, vulnerability verification, malware, and artificial intelligence.